TP S3 - Kubernetes : Scaling & Networking

Cloud & Data Engineering — EPITA IA Institut 2025/2026 | Formateur : Hamza Abid (MCT)

Objectifs pedagogiques

A l'issue de ce TP, vous serez capable de :

1. **Scaler** manuellement et automatiquement une application Kubernetes
2. **Observer** la distribution de charge entre pods et comprendre le load balancing
3. **Installer** et configurer un Ingress Controller NGINX
4. **Creer** des regles Ingress avec path-based routing
5. **Effectuer** un rolling update sans coupure et un rollback
6. **Configurer** un HPA (Horizontal Pod Autoscaler) et observer l'autoscaling
7. **Deployer** une architecture multi-services avec routing intelligent

---

Prerequis

- minikube demarre : `minikube start`
- kubectl configure et fonctionnel
- Docker Desktop actif
- Avoir suivi les notions du TP S2 (Deployment Flask API)

Verifier l'etat du cluster :

```
# Verifie que le noeud minikube est pret (status = Ready)
kubectl get nodes

# Liste tous les pods du cluster, y compris les pods systeme
kubectl get pods --all-namespaces
```

Resultat attendu :

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   ...   v1.xx.x
```

Structure du TP

Partie	Sujet
Preparation	Application de base (Deployment + Service)
Partie 1	Scaling manuel et observation du load balancing
Partie 2	Ingress Controller et regles de routage
Partie 3	Rolling Update et Rollback
Partie 4	Horizontal Pod Autoscaler (HPA)
Challenge	Architecture multi-services avec routing Ingress
Bonus	Network Policy (isolation reseau)

Preparation - Application de base

Rappel : Deployment + Service

Un **Deployment** gere un ensemble de pods identiques (replicas). Il s'assure que le nombre souhaite de pods tourne en permanence.
Un **Service** expose ces pods sur le reseau interne du cluster. Il sert de point d'entree stable (adresse DNS fixe) vers les pods, meme si ceux-ci sont recrees.

Creez le fichier `flask-deployment.yaml` qui servira de base pour tout le TP :

```yaml
# fichier: flask-deployment.yaml
# Ce fichier cree deux ressources Kubernetes : un Deployment et un Service.

apiVersion: apps/v1
kind: Deployment                # Un Deployment gere un ensemble de pods identiques
metadata:
  name: flask-api               # Nom du Deployment (reference par ce nom)
  labels:
    app: flask-api              # Label pour identifier cette application
spec:
  replicas: 1                   # Nombre de copies (pods) au depart
  selector:
    matchLabels:
      app: flask-api            # Le Deployment gere les pods ayant ce label
  strategy:
    type: RollingUpdate         # Strategie de mise a jour progressive
```

```yaml
    rollingUpdate:
      maxUnavailable: 1         # Max 1 pod indisponible pendant la mise a jour
      maxSurge: 1               # Max 1 pod supplementaire pendant la mise a jour
  template:
    metadata:
      labels:
        app: flask-api          # Chaque pod cree aura ce label
    spec:
      containers:
      - name: flask-api
        image: nginx:alpine     # Serveur web leger comme application de test
        ports:
        - containerPort: 80     # Le conteneur ecoute sur le port 80 (HTTP)
        env:
        - name: POD_NAME        # Variable contenant le nom du pod
          valueFrom:
            fieldRef:
              fieldPath: metadata.name  # Kubernetes injecte le nom du pod
        lifecycle:
          postStart:             # Hook execute apres le demarrage du conteneur
            exec:
              command: ["/bin/sh", "-c",
                "echo Hello depuis le pod $POD_NAME > /usr/share/nginx/html/index.html"]
              # Ecrit un message personnalise avec le nom du pod
        resources:
          requests:              # Ressources minimum demandees
            cpu: "100m"          # 100 millicores = 10% d'un coeur CPU
            memory: "64Mi"       # 64 megaoctets de memoire
          limits:                # Ressources maximum autorisees
            cpu: "250m"          # 250 millicores = 25% d'un coeur CPU
            memory: "128Mi"      # 128 megaoctets de memoire
        volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html  # Volume pour la page HTML
      volumes:
      - name: html
        emptyDir: {}             # Volume temporaire vide, partage dans le pod
---
apiVersion: v1
kind: Service                    # Expose les pods sur le reseau interne
metadata:
  name: flask-api-service        # Nom du Service (= adresse DNS interne)
spec:
  selector:
    app: flask-api               # Redirige vers les pods ayant ce label
  ports:
  - port: 80                     # Port expose par le Service
    targetPort: 80               # Port du conteneur cible
  type: ClusterIP                # Accessible uniquement depuis le cluster
```

```bash
# Applique le manifest : cree le Deployment et le Service
kubectl apply -f flask-deployment.yaml

# Verifie que le pod est cree et en cours d'execution
kubectl get pods
```

```
# Verifie que le Service est cree
kubectl get service
```

Resultat attendu :

```
NAME                        READY   STATUS    RESTARTS   AGE
flask-api-xxxxxxxxxx-xxxxx  1/1     Running   0          30s

NAME                TYPE        CLUSTER-IP      PORT(S)
flask-api-service   ClusterIP   10.96.xxx.xxx   80/TCP
```

> **Note :** Le conteneur `nginx:alpine` genere dynamiquement sa page d'accueil via un hook `postStart` qui ecrit le nom du pod dans `/usr/share/nginx/html/index.html`. Cela permet d'identifier quel pod repond a chaque requete — parfait pour observer la distribution de charge.

Partie 1 - Scaling manuel et observation du load balancing

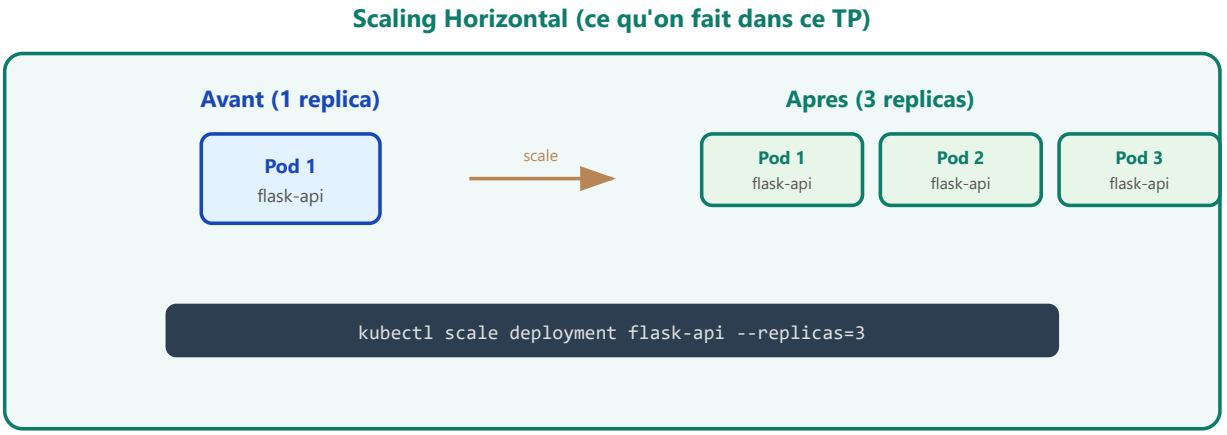
C'est quoi le scaling ?

Imaginez un restaurant. Un seul serveur peut gerer 10 tables. Si 50 clients arrivent en meme temps, il faut embaucher plus de serveurs. C'est exactement ce que fait le **scaling** dans Kubernetes.

Il existe deux types de scaling :

- **Scaling vertical** (scale up) : donner plus de puissance a une machine existante (plus de CPU, plus de RAM). Comme donner un plateau plus grand a votre unique serveur — il y a une limite physique.
- **Scaling horizontal** (scale out) : ajouter plus de machines identiques. Comme embaucher 4 serveurs supplementaires — c'est la methode preferee dans Kubernetes car elle n'a pas de limite theorique.

Dans cette partie, on fait du **scaling horizontal** : augmenter le nombre de pods (copies) de notre application.



1.1 - Scaler a 3 replicas via la commande

```
# Passe le Deployment de 1 a 3 replicas (3 copies identiques)
kubectl scale deployment flask-api --replicas=3

# Observer la creation des pods en temps reel (-w = watch)
kubectl get pods -w
# Appuyer sur Ctrl+C quand les 3 pods sont Running
```

Resultat attendu :

```
NAME                          READY   STATUS    RESTARTS   AGE
flask-api-xxxxxxxxxx-aaaaa    1/1     Running   0          5m
flask-api-xxxxxxxxxx-bbbbb    1/1     Running   0          10s
flask-api-xxxxxxxxxx-ccccc    1/1     Running   0          10s
```

> **Question :** Combien de ReplicaSets voyez-vous ? Pourquoi ?
> *Indice : un ReplicaSet est le "gardien" qui s'assure que le bon nombre de pods tourne. Il est cree automatiquement par le Deployment.*

```
# Liste les ReplicaSets (le composant qui maintient le nombre de pods)
kubectl get replicasets

# Affiche les details d'un ReplicaSet specifique
kubectl describe replicaset <nom-du-rs>
```

1.2 - Scaler via le manifest YAML

Il existe deux facons de scaler :

- **Imperatif** (commande) : `kubectl scale` — rapide, mais pas trace dans le code
- **Declaratif** (YAML) : modifier le fichier et faire `kubectl apply` — recommande en production car versionnable avec Git

Modifier `flask-deployment.yaml` : changer `replicas: 1` en `replicas: 3`, puis :

```
# Applique le fichier modifie ; Kubernetes detecte le changement et ajuste
kubectl apply -f flask-deployment.yaml

# Verifie le nombre de pods
kubectl get pods
```

> **Bonne pratique :** En production, on privilegie toujours l'approche declarative (YAML + Git) pour pouvoir tracer et reproduire les changements. L'approche imperative est utile pour le debug et les tests rapides.

1.3 - Observer la distribution de charge

On va maintenant prouver que Kubernetes repartit le trafic entre les 3 pods.

> **Note multi-OS :** La commande `port-forward` bloque le terminal. Ouvrez **un terminal dedie** pour le port-forward et travaillez dans un second terminal pour les commandes `curl`.

Terminal 1 (dedie au port-forward) :

```
# Redirige le port 8080 de votre machine vers le port 80 du Service
# Le trafic arrive au Service, qui le distribue entre les 3 pods
kubectl port-forward service/flask-api-service 8080:80
```

Terminal 2 (pour tester) :

```
# Fait 10 requetes HTTP et affiche les reponses
# Chaque reponse contiendra le nom du pod qui a repondu
for i in $(seq 1 10); do curl http://localhost:8080; echo; done
```

Observation attendue : Chaque reponse provient d'un pod different. Kubernetes repartit la charge selon un mecanisme appele **round-robin** (chacun son tour).

```
Hello depuis le pod flask-api-xxxxxxxxxx-aaaaa
Hello depuis le pod flask-api-xxxxxxxxxx-bbbbb
Hello depuis le pod flask-api-xxxxxxxxxx-ccccc
Hello depuis le pod flask-api-xxxxxxxxxx-aaaaa
...
```

Comment fonctionne le load balancing ?

Quand vous envoyez une requete au **Service**, celui-ci ne la traite pas lui-meme. Il maintient une liste des pods cibles (les **Endpoints**) et redirige chaque requete vers un pod different. C'est `kube-proxy`, un composant systeme de Kubernetes, qui gere cette distribution via des regles iptables.

```
# Affiche les 5 dernieres lignes de logs de chaque pod flask-api
kubectl get pods -l app=flask-api --no-headers | awk '{print $1}' | while read pod; do
  echo "=== Logs de $pod ===";
  kubectl logs $pod --tail=5;
done

# Voir les Endpoints du Service (= IPs des pods cibles)
kubectl get endpoints flask-api-service
```

1.4 - Simuler la panne d'un pod

Un des grands avantages de Kubernetes : la **self-healing** (auto-reparation). Si un pod tombe, le ReplicaSet en recree un automatiquement.

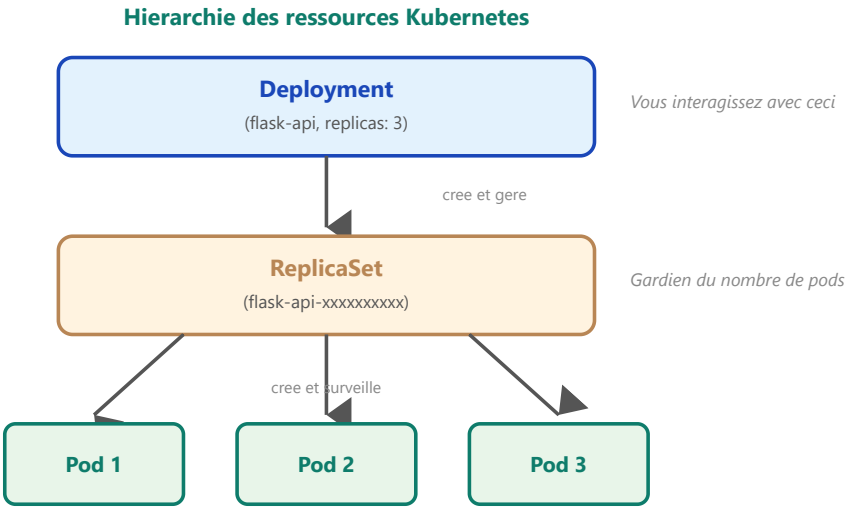
```
# Supprime un pod manuellement (remplacez <nom-du-pod> par un vrai nom)
kubectl delete pod <nom-du-pod>

# Observe la recreation automatique par le ReplicaSet
kubectl get pods -w
```

Resultat attendu : Le ReplicaSet detecte qu'il manque un pod et en cree un nouveau automatiquement. Observez la colonne AGE du nouveau pod.

> **Question :** Combien de temps le ReplicaSet met-il a recreer le pod ?
> *Indice : observez la colonne AGE du nouveau pod dans la sortie de* `kubectl get pods -w`.

1.5 - Comprendre le lien Deployment → ReplicaSet → Pod




```
# Visualiser cette hierarchie
kubectl get deployment flask-api
kubectl get replicasets
kubectl get pods -l app=flask-api

# Voir les details avec les events
kubectl describe deployment flask-api
```

Question de reflexion : Quand vous scalez de 1 a 3 replicas, Kubernetes cree-t-il un nouveau ReplicaSet ou modifie-t-il l'existant ? Et lors d'un Rolling Update (Partie 3), que se passe-t-il au niveau des ReplicaSets ?
Indice : pensez a la difference entre changer le nombre de copies d'une meme version et deployer une nouvelle version.

Checkpoint Partie 1 : A ce stade, vous devriez avoir :

- 3 pods `flask-api-xxx` en statut Running
- La boucle curl affichant des reponses provenant de differents pods (noms differents)
- Constate que le ReplicaSet recree un pod apres suppression manuelle
- Compris la hierarchie Deployment → ReplicaSet → Pod

Partie 2 - Ingress Controller et Regles de routage

C'est quoi un Ingress ?

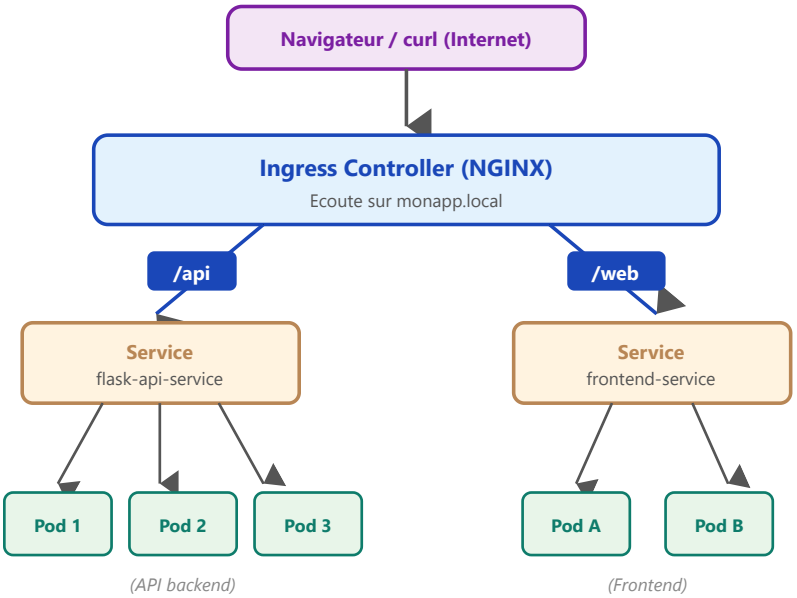
Imaginez un hotel. Quand vous arrivez a la reception, vous dites "je vais au restaurant" ou "je vais a la piscine", et le receptionniste vous indique le chemin.

Un **Ingress** fonctionne de la meme maniere dans Kubernetes :

- Le **receptionniste** = l'Ingress Controller (un pod NGINX special)
- Les **clients** = les requetes HTTP venant d'Internet
- Les **espaces** = vos Services Kubernetes
- Les **panneaux indicateurs** = les regles Ingress (ex: `/api` → Service API, `/web` → Service Frontend)

Sans Ingress, il faudrait exposer chaque Service individuellement avec un port different (NodePort). Avec 20 microservices, cela deviendrait ingerant. L'Ingress centralise tous les acces sur un seul point d'entree.

Schema : Ingress — Routing multi-path



2.1 - Installer NGINX Ingress Controller sur minikube

```
# Active l'addon ingress de minikube (installe le controller NGINX)
minikube addons enable ingress

# Attend que le pod du controller soit pret (timeout apres 120 secondes)
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=120s

# Verifie que le controller tourne dans son namespace dedie
kubectl get pods -n ingress-nginx
```

```
# Verifie le Service du controller
kubectl get service -n ingress-nginx
```

Resultat attendu :

```
NAME                                  READY   STATUS    RESTARTS   AGE
ingress-nginx-controller-xxxxxxxxxx-x   1/1     Running   0          60s
```

> **Namespace :** Le controller Ingress tourne dans son propre namespace `ingress-nginx`, isole du namespace `default` ou tournent vos pods applicatifs. C'est une bonne pratique de separation des responsabilites.

2.2 - Recuperer l'IP de minikube et configurer le DNS local

```
# Affiche l'adresse IP de la VM minikube
minikube ip
# Exemple de sortie : 192.168.49.2
```

Ajouter une entree dans votre fichier hosts pour associer `monapp.local` a l'IP de minikube :

Linux / Mac :

```
# Ajoute l'entree dans /etc/hosts (necessite les droits admin)
echo "$(minikube ip)  monapp.local" | sudo tee -a /etc/hosts
```

Windows (PowerShell en administrateur) :

```
# Etape 1 : Recuperer l'IP de minikube
$MINIKUBE_IP = minikube ip

# Etape 2 : Ajouter l'entree dans le fichier hosts
Add-Content -Path C:\Windows\System32\drivers\etc\hosts -Value "$MINIKUBE_IP monapp.local"
```

> **Verification :** Tapez `ping monapp.local` pour verifier que l'adresse est resolue vers l'IP minikube.

Pourquoi modifier le fichier hosts ?

En production, on utilise un vrai nom de domaine (ex: `api.monentreprise.com`) pointe vers un load balancer. Ici, en local, le fichier `hosts` simule un DNS en associant `monapp.local` a l'IP de minikube. Ainsi, quand vous tapez `curl http://monapp.local`, la requete arrive bien sur le cluster.

2.3 - Creer la ressource Ingress

Creez le fichier `ingress-api.yaml` :

```yaml
# fichier: ingress-api.yaml
# Cette ressource Ingress definit les regles de routage HTTP.

apiVersion: networking.k8s.io/v1
kind: Ingress                          # Configure le routage HTTP
metadata:
  name: flask-api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    # Reecrit l'URL : /api/xxx devient /xxx avant envoi au Service
spec:
  ingressClassName: nginx              # Utilise le controller NGINX
  rules:
  - host: monapp.local                 # Nom de domaine declencheur
    http:
      paths:
      - path: /api                     # Quand l'URL commence par /api...
        pathType: Prefix               # ... (tout ce qui commence par /api)
        backend:
          service:
            name: flask-api-service    # ... redirige vers ce Service
            port:
              number: 80               # ... sur le port 80
```

```
# Cree la ressource Ingress dans le cluster
kubectl apply -f ingress-api.yaml

# Verifie que l'Ingress est cree (ADDRESS peut mettre 1-2 min a apparaitre)
kubectl get ingress

# Affiche les details de la regle Ingress
kubectl describe ingress flask-api-ingress
```

Resultat attendu :

```
NAME                CLASS   HOSTS          ADDRESS        PORTS   AGE
flask-api-ingress   nginx   monapp.local   192.168.49.2   80      30s
```

2.4 - Tester l'acces via Ingress

```
# Teste l'acces via le nom de domaine et le path /api
curl http://monapp.local/api

# Fait 6 requetes pour observer la distribution entre les pods
for i in $(seq 1 6); do curl http://monapp.local/api; echo; done
```

Resultat attendu : Les reponses alternent entre les 3 pods.

```
Hello depuis le pod flask-api-xxxxxxxxxx-aaaaa
Hello depuis le pod flask-api-xxxxxxxxxx-bbbbb
Hello depuis le pod flask-api-xxxxxxxxxx-ccccc
Hello depuis le pod flask-api-xxxxxxxxxx-aaaaa
Hello depuis le pod flask-api-xxxxxxxxxx-bbbbb
Hello depuis le pod flask-api-xxxxxxxxxx-ccccc
```

2.5 - Verifier les logs du Controller Ingress

```
# Affiche les 20 dernieres lignes de logs du controller Ingress
kubectl logs -n ingress-nginx -l app.kubernetes.io/component=controller --tail=20
```

Question de reflexion : L'Ingress Controller est lui-meme un Pod. Pourquoi est-il preferable d'utiliser un Ingress plutot que d'exposer chaque Service en NodePort dans un cluster de production avec 20 microservices ?
Indice : pensez au nombre de ports a gerer, a la configuration DNS, et a la possibilite de faire du HTTPS avec un seul certificat.

Mini-exercice : Que se passe-t-il si vous faites `curl http://monapp.local/autre-chose` ? Pourquoi ?
Testez-le et expliquez le code HTTP retourne.

Checkpoint Partie 2 : A ce stade, vous devriez avoir :

- Le pod `ingress-nginx-controller` en statut Running dans le namespace `ingress-nginx`
- L'entree `monapp.local` ajoutee dans le fichier hosts
- `curl http://monapp.local/api` retournant une reponse de vos pods
- Compris le role de l'Ingress comme "reception" du cluster

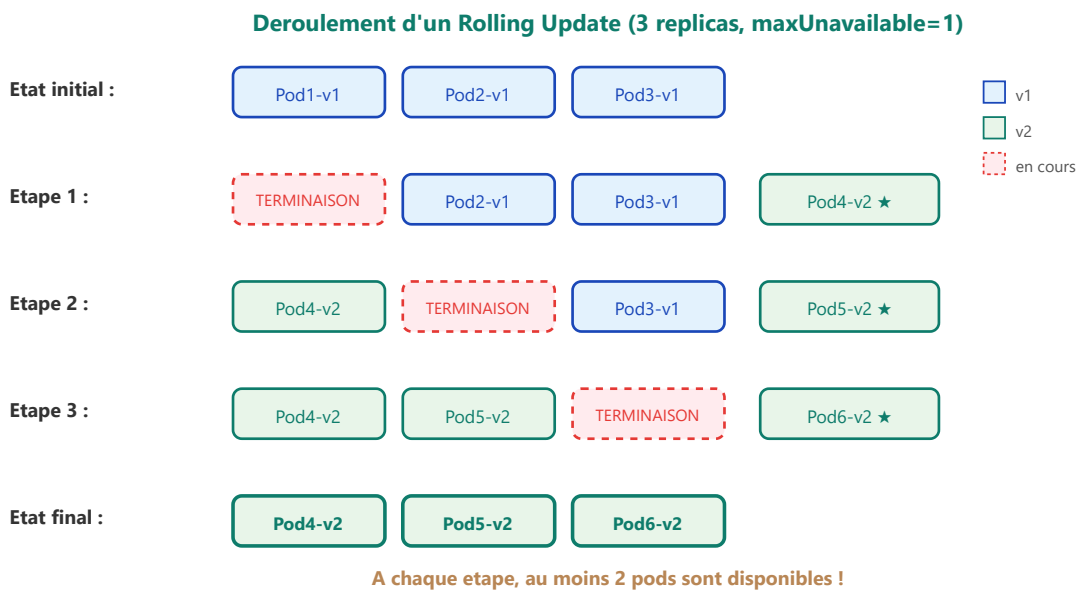
Partie 3 - Rolling Update et Rollback

C'est quoi un Rolling Update ?

Imaginez que vous avez une boutique avec 3 vendeurs. Vous voulez changer leur uniforme. Plutot que de fermer la boutique pour les habiller tous en meme temps (ce qui perdrait des clients), vous les changez **un par un** : pendant que le vendeur 1 se change, les vendeurs 2 et 3 continuent de servir les clients.

C'est exactement ce que fait le **Rolling Update** dans Kubernetes :

- Les pods sont remplaces **un par un** (ou selon la strategie configuree)
- Il y a toujours des pods disponibles pour repondre aux requetes
- **Zero interruption de service** (zero downtime)
- Si la nouvelle version a un probleme, on peut revenir en arriere avec un **rollback**



3.1 - Effectuer un Rolling Update

On va simuler une mise a jour de version en changeant le texte retourne par l'application. Creez le fichier `flask-deployment-v2.yaml` :

```
# fichier: flask-deployment-v2.yaml
# Meme Deployment mais avec un message different pour simuler une v2.

apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-api              # Meme nom = mise a jour du Deployment existant
  annotations:
    kubernetes.io/change-cause: "Update vers v2 - nouveau message"
    # Apparaitra dans l'historique des rollouts
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flask-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1        # Max 1 pod indisponible pendant la mise a jour
      maxSurge: 1              # Max 1 pod supplementaire temporaire
  template:
    metadata:
      labels:
```

```yaml
        app: flask-api
    spec:
      containers:
      - name: flask-api
        image: nginx:alpine    # On garde la meme image nginx
        ports:
        - containerPort: 80
        env:
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        lifecycle:
          postStart:
            exec:
              command: ["/bin/sh", "-c",
                "echo 'VERSION 2 - Pod: '$(hostname) > /usr/share/nginx/html/index.html"]
              # Le message change : "VERSION 2" au lieu de "Hello depuis le pod"
              # C'est ce changement qui declenche le Rolling Update
        resources:
          requests:
            cpu: "100m"
            memory: "64Mi"
          limits:
            cpu: "250m"
            memory: "128Mi"
        volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
      volumes:
      - name: html
        emptyDir: {}
```

> **Point important :** On garde `nginx:alpine` comme image. Si on changeait pour une image ecoutant sur un port different (ex: port 5678), le Service resterait configure sur le port 80 et le rolling update **casserait l'application**. Toujours verifier la compatibilite des ports lors d'un changement d'image.

```bash
# Applique la v2 (Kubernetes detecte le changement de template)
kubectl apply -f flask-deployment-v2.yaml

# Suit la progression du rolling update en temps reel
kubectl rollout status deployment/flask-api
```

Dans un autre terminal, testez en continu pendant le rolling update :

```bash
# Fait une requete toutes les 0.5 secondes pour observer la transition
while true; do curl http://monapp.local/api 2>/dev/null; echo; sleep 0.5; done
# Appuyer sur Ctrl+C pour arreter
```

Observation : Pendant le rolling update, vous verrez alternativement des reponses "VERSION 2" et l'ancien message "Hello depuis le pod" — preuve qu'il n'y a **pas de coupure de service**.

```
Hello depuis le pod flask-api-xxxxxxxxxx-aaaaa
VERSION 2 - Pod: flask-api-yyyyyyyyyy-bbbbb
Hello depuis le pod flask-api-xxxxxxxxxx-ccccc
VERSION 2 - Pod: flask-api-yyyyyyyyyy-ddddd
VERSION 2 - Pod: flask-api-yyyyyyyyyy-eeeee
```

3.2 - Verifier l'historique des rollouts

```
# Affiche toutes les revisions du Deployment (historique des mises a jour)
kubectl rollout history deployment/flask-api

# Affiche les details de la revision 2 (la v2 qu'on vient de deployer)
kubectl rollout history deployment/flask-api --revision=2
```

Resultat attendu :

```
REVISION  CHANGE-CAUSE
1         <none>
2         Update vers v2 - nouveau message
```

> **Mini-exercice :** Verifiez le nombre de ReplicaSets avec `kubectl get replicasets`. Combien en voyez-vous maintenant et pourquoi ? Quel est l'etat de l'ancien ReplicaSet ?
> *Indice : Kubernetes garde l'ancien ReplicaSet (avec 0 replicas) pour permettre un rollback rapide.*

3.3 - Effectuer un Rollback

Le rollback permet de revenir instantanement a une version precedente — essentiel quand un deploiement introduit un bug en production.

```
# Revient a la version precedente (revision 1)
kubectl rollout undo deployment/flask-api

# Suit la progression du rollback
kubectl rollout status deployment/flask-api

# Verifie que l'ancienne version est revenue
curl http://monapp.local/api
```

Resultat attendu : Le message revient a "Hello depuis le pod ..." au lieu de "VERSION 2".

3.4 - Rollback vers une revision specifique

```
# Affiche les revisions disponibles
kubectl rollout history deployment/flask-api

# Rollback vers la revision 2 (la v2)
kubectl rollout undo deployment/flask-api --to-revision=2
```

```
# Verifie que la v2 est de retour
curl http://monapp.local/api
```

Question : Quelle est la difference entre `rollout undo` et `rollout undo --to-revision=X` ?
Indice : `rollout undo` sans option revient toujours a la version juste precedente. Avec `--to-revision=X`, on peut choisir n'importe quelle version de l'historique.

Question de reflexion : En production, vous deployez la v3 d'une application et un bug critique est detecte 2 heures plus tard. La v2 etait stable. Quelle commande executez-vous ? Pourquoi le rollback est-il plus rapide que de re-deployer la v2 depuis le code source ?
Indice : le rollback reutilise l'ancien ReplicaSet qui est deja present dans le cluster.

Checkpoint Partie 3 : A ce stade, vous devriez avoir :

- Effectue un rolling update de v1 vers v2 sans interruption de service
- Observe les deux messages coexister pendant la transition
- Effectue un rollback vers la v1 puis un rollback cible vers la v2
- Compris le role des ReplicaSets dans le mecanisme de rollback

---

Partie 4 - Horizontal Pod Autoscaler (HPA)

C'est quoi un HPA ?

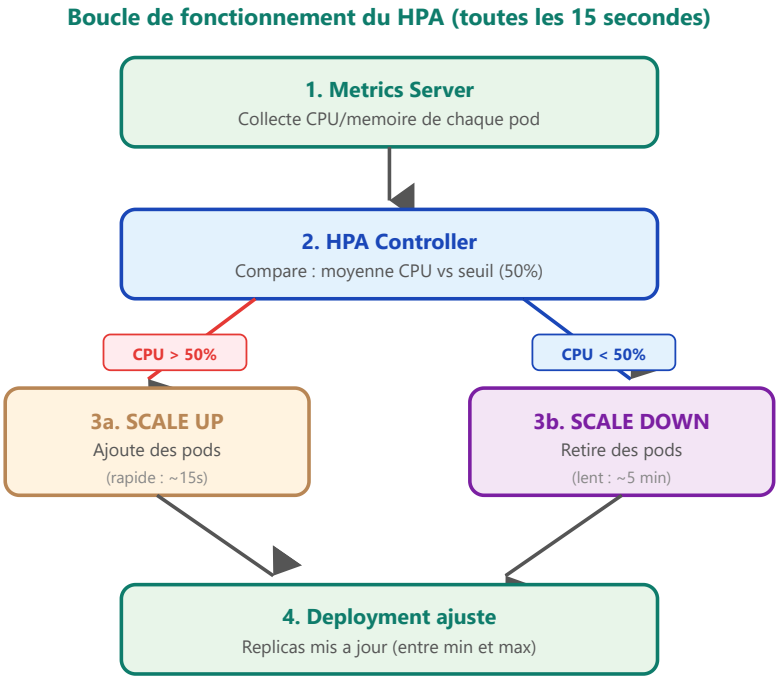
Le **HPA** (Horizontal Pod Autoscaler) ajuste **automatiquement** le nombre de pods en fonction de la charge.

Imaginez un supermarche :

- En temps normal, 2 caisses suffisent
- Le samedi a 17h, quand la file s'allonge, le manager ouvre des caisses supplementaires
- Quand l'affluence diminue, il referme les caisses inutiles

Le HPA fait exactement cela :

- Il surveille l'utilisation **CPU** (ou d'autres metriques) des pods
- Si la moyenne depasse le seuil configure (ex: 50% CPU), il **ajoute des pods** (scale up)
- Si la moyenne descend bien en dessous du seuil, il **reduit le nombre de pods** (scale down)
- Il respecte toujours les limites **min** et **max** que vous definissez



4.1 - Activer metrics-server

Le HPA a besoin de **metriques** (donnees CPU/memoire) pour prendre ses decisions. Le composant `metrics-server` collecte ces donnees.

```
# Active metrics-server sur minikube
minikube addons enable metrics-server

# Patientez 1-2 minutes puis verifiez que les metriques sont disponibles
kubectl top nodes
# Affiche l'utilisation CPU et memoire du noeud

kubectl top pods
# Affiche l'utilisation CPU et memoire de chaque pod
```

Resultat attendu :

```
NAME       CPU(cores)   MEMORY(bytes)
minikube   250m         1200Mi

NAME                        CPU(cores)   MEMORY(bytes)
flask-api-xxxxxxxxxx-aaaaa   1m           5Mi
flask-api-xxxxxxxxxx-bbbbb   1m           5Mi
flask-api-xxxxxxxxxx-ccccc   1m           5Mi
```

> **Erreur courante :** Si `kubectl top` renvoie "Metrics API not available", `metrics-server` n'a pas encore fini de demarrer. Patientez 2 minutes et reessayez.

4.2 - Configurer le HPA

Option A : via la commande kubectl (rapide pour tester) :

```
# Cree un HPA qui gere automatiquement le nombre de pods flask-api
# - minimum 2 pods (meme sans charge)
# - maximum 5 pods (pour ne pas surcharger le cluster)
# - seuil : si la moyenne CPU depasse 50%, on ajoute des pods
kubectl autoscale deployment flask-api \
  --min=2 \
  --max=5 \
  --cpu-percent=50

# Verifie la creation du HPA
kubectl get hpa

# Affiche les details du HPA
kubectl describe hpa flask-api
```

Option B : via manifest YAML (recommande pour la reproductibilite) :

```
# fichier: hpa.yaml
# Le HPA surveille l'utilisation CPU et ajuste le nombre de pods.

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: flask-api-hpa
spec:
  scaleTargetRef:              # Le Deployment a surveiller
    apiVersion: apps/v1
    kind: Deployment
    name: flask-api
  minReplicas: 2               # Jamais moins de 2 (haute disponibilite)
  maxReplicas: 5               # Jamais plus de 5 (protection ressources)
  metrics:
  - type: Resource
    resource:
      name: cpu                # Metrique surveillee : utilisation CPU
      target:
        type: Utilization
        averageUtilization: 50 # Seuil : 50% CPU en moyenne
```

```
# Applique le manifest HPA
kubectl apply -f hpa.yaml

# Verifie l'etat du HPA
kubectl get hpa flask-api-hpa
```

Resultat attendu :

```
NAME            REFERENCE               TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
flask-api-hpa   Deployment/flask-api    2%/50%    2         5         2          30s
```

> **Note :** Si la colonne TARGETS affiche `<unknown>/50%`, metrics-server n'a pas encore collecte les donnees. Patientez 1-2 minutes.

Comment le HPA calcule le nombre de replicas ?

La formule est : `replicas_voulus = ceil(replicas_actuels × (CPU_actuelle / CPU_cible))`

Exemple : Si on a 2 pods a 80% CPU et le seuil est 50% :
`ceil(2 × 80/50) = ceil(3.2) = 4 replicas`

Le HPA ne depassera jamais `maxReplicas` et ne descendra jamais sous `minReplicas`.

4.3 - Observer le HPA au repos

```
# Observe le HPA en temps reel
kubectl get hpa flask-api-hpa --watch
```

Au debut, vous devriez voir TARGETS afficher `2%/50%` et REPLICAS: 2 (le minimum configure).

4.4 - Generer de la charge (stress test)

Ouvrir un second terminal et lancer un generateur de charge :

```
# Cree un pod temporaire qui envoie des requetes en boucle vers notre Service
# Cela va augmenter l'utilisation CPU des pods flask-api
kubectl run load-generator \
  --image=busybox:1.28 \
  --restart=Never \
  -- /bin/sh -c "while true; do wget -q -O- http://flask-api-service; done"

# Verifie que le pod generateur de charge est en cours d'execution
kubectl get pod load-generator
```

Dans le premier terminal, observer le scaling :

> **Note multi-OS :** La commande `watch` n'est pas disponible nativement sur Windows et Mac. Utilisez cette alternative compatible partout :

```
# Affiche le HPA et les pods toutes les 2 secondes (Windows/Mac/Linux)
while true; do kubectl get hpa && echo "---" && kubectl get pods -l app=flask-api; sleep 2
```

Sur Linux avec `watch` installe :

```
watch -n 2 'kubectl get hpa && echo "---" && kubectl get pods -l app=flask-api'
```

Resultat attendu : Apres 1-2 minutes, le HPA augmente le nombre de replicas (scale up vers 4 ou 5 pods).

```
NAME            REFERENCE               TARGETS    MINPODS   MAXPODS   REPLICAS
flask-api-hpa   Deployment/flask-api    78%/50%    2         5         5
---
NAME                         READY   STATUS    RESTARTS   AGE
flask-api-xxxxxxxxxx-aaaaa   1/1     Running   0          10m
flask-api-xxxxxxxxxx-bbbbb   1/1     Running   0          10m
flask-api-xxxxxxxxxx-ccccc   1/1     Running   0          45s
flask-api-xxxxxxxxxx-ddddd   1/1     Running   0          45s
flask-api-xxxxxxxxxx-eeeee   1/1     Running   0          30s
```

4.5 - Arreter la charge et observer le scale-down

```
# Supprime le pod generateur de charge
kubectl delete pod load-generator

# Observe le scale-down (plus lent, environ 5 minutes par defaut)
kubectl get hpa flask-api-hpa --watch
```

Resultat attendu : Apres quelques minutes, le nombre de replicas redescend progressivement a 2 (le minimum configure).

> **Question :** Pourquoi le scale-down est-il plus lent que le scale-up ?
> *Indice : imaginez que la charge revient brutalement juste apres avoir reduit le nombre de pods. Si le scale-down etait instantane, l'application serait temporairement surchargee le temps de recreer les pods. Le delai de stabilisation (5 minutes par defaut) protege contre ces "faux calmes".*

> **Question de reflexion :** Vous etes en production et le HPA oscille sans cesse entre 2 et 5 pods (il scale up, puis scale down, puis scale up...). Quel parametre ajusteriez-vous pour stabiliser le comportement ?
> *Indice : pensez au seuil CPU (50%), aux limites min/max, et au delai de stabilisation.*

4.6 - Script de stress test alternatif

Si busybox n'est pas disponible, utilisez ce pod de test :

```
# fichier: load-generator.yaml
# Alternative avec alpine + curl
apiVersion: v1
kind: Pod
metadata:
  name: load-generator
spec:
  containers:
  - name: load-generator
    image: alpine:latest
```

```
    command:
    - /bin/sh
    - -c
    - |
      apk add --no-cache curl -q   # Installe curl
      while true; do               # Boucle infinie de requetes
        curl -s http://flask-api-service > /dev/null
      done
  restartPolicy: Never
```

```
# Applique et verifiez
kubectl apply -f load-generator.yaml
kubectl logs load-generator --follow
```

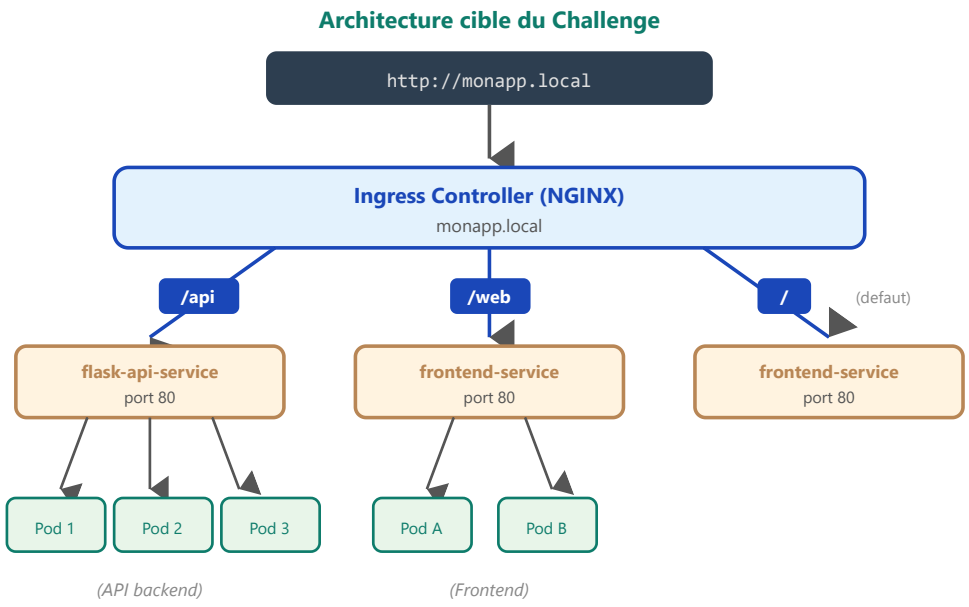
Checkpoint Partie 4 : A ce stade, vous devriez avoir :

- Le HPA `flask-api-hpa` configure avec minReplicas: 2 et maxReplicas: 5
- Observe le passage a 4 ou 5 replicas pendant la generation de charge (TARGETS > 50%)
- Le scale-down progressif vers 2 replicas apres suppression du generateur
- Compris la formule de calcul et l'asymetrie scale-up rapide / scale-down lent

---

Challenge - Architecture multi-services avec Ingress

Objectif : Deployer un service frontend (nginx) et configurer le routing par path avec Ingress. Cela simule une architecture reelle ou un frontend web et une API backend sont accessibles depuis le meme nom de domaine.



Challenge 1 - Deployer un service frontend

Creez le fichier `frontend-deployment.yaml` :

```yaml
# fichier: frontend-deployment.yaml
# Deploie un frontend simple (page HTML statique) avec 2 replicas.

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  replicas: 2                  # 2 copies pour la haute disponibilite
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: frontend
        image: nginx:alpine
        ports:
        - containerPort: 80
        volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
      initContainers:          # S'execute AVANT le conteneur principal
      - name: init-html
        image: busybox:1.28
        command:
        - /bin/sh
        - -c
        - echo "<h1>Frontend EPITA - Kubernetes TP S3</h1>" > /html/index.html
        volumeMounts:
        - name: html
          mountPath: /html
      volumes:
      - name: html
        emptyDir: {}           # Partage entre initContainer et nginx
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 80
  type: ClusterIP
```

```
# Deploie le frontend
kubectl apply -f frontend-deployment.yaml

# Verifie que les 2 pods frontend sont Running
kubectl get pods -l app=frontend

# Verifie que le Service est cree
kubectl get service frontend-service
```

Resultat attendu :

```
NAME                        READY   STATUS    RESTARTS   AGE
frontend-xxxxxxxxxx-aaaaa   1/1     Running   0          30s
frontend-xxxxxxxxxx-bbbbb   1/1     Running   0          30s
```

initContainer vs postStart : quelle difference ?

Dans le Deployment de l'API, on a utilise un **postStart hook** (s'execute apres le demarrage du conteneur principal). Ici, on utilise un **initContainer** (s'execute avant le conteneur principal).

- **initContainer** : demarre, fait son travail, puis se termine. Le conteneur principal ne demarre qu'apres. Ideal pour la preparation (migration de BDD, telechargement de config, etc.).
- **postStart** : s'execute en parallele du demarrage du conteneur principal. Moins fiable pour les dependances.

Challenge 2 - Configurer l'Ingress multi-path

Creez le fichier `ingress-multipath.yaml` :

```
# fichier: ingress-multipath.yaml
# Ingress avec 3 regles de routage basees sur le chemin (path).

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multipath-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    # Capture le 2eme groupe regex et le passe au backend
    # Exemple : /api/users devient /users
spec:
  ingressClassName: nginx
  rules:
  - host: monapp.local
    http:
      paths:
      - path: /api(/|$)(.*)              # Tout ce qui commence par /api
        pathType: ImplementationSpecific
        backend:
```

```
          service:
            name: flask-api-service      # → vers l'API backend
            port:
              number: 80
      - path: /web(/|$)(.*)              # Tout ce qui commence par /web
        pathType: ImplementationSpecific
        backend:
          service:
            name: frontend-service       # → vers le frontend
            port:
              number: 80
      - path: /                          # Tout le reste (racine)
        pathType: Prefix
        backend:
          service:
            name: frontend-service       # → par defaut, vers le frontend
            port:
              number: 80
```

```
# Supprime l'ancien ingress (meme host = conflit possible)
kubectl delete ingress flask-api-ingress

# Applique le nouvel ingress multi-path
kubectl apply -f ingress-multipath.yaml

# Verifie la creation
kubectl get ingress

# Affiche les details des regles de routage
kubectl describe ingress multipath-ingress
```

Challenge 3 - Tester le routing

```
# Teste le path /web → redirige vers le frontend (nginx)
curl http://monapp.local/web
# Attendu : <h1>Frontend EPITA - Kubernetes TP S3</h1>

# Teste le path /api → redirige vers flask-api
curl http://monapp.local/api
# Attendu : VERSION 2 - Pod: flask-api-...

# Teste la racine / → redirige vers le frontend (par defaut)
curl http://monapp.local/
# Attendu : <h1>Frontend EPITA - Kubernetes TP S3</h1>
```

Resultat attendu : Chaque path mene vers un Service different, exactement comme un receptionniste d'hotel qui redirige les visiteurs vers differentes zones.

Challenge 4 - Validation finale

Verifiez l'etat complet du cluster :

```
# Vue d'ensemble complete du cluster
echo "=== PODS ===="
kubectl get pods -o wide

echo "=== SERVICES ===="
kubectl get services

echo "=== INGRESS ===="
kubectl get ingress

echo "=== HPA ===="
kubectl get hpa

echo "=== REPLICASETS ===="
kubectl get replicasets
```

Checkpoint Challenge : A ce stade, vous devriez avoir :

- 2 pods frontend en statut Running
- `curl http://monapp.local/web` → page frontend
- `curl http://monapp.local/api` → reponse API
- `curl http://monapp.local/` → page frontend (defaut)
- Compris la difference entre `Prefix`, `ImplementationSpecific` et le mecanisme de rewrite

---

Bonus - Network Policy (isolation reseau)

C'est quoi une Network Policy ?

Une **Network Policy** est un pare-feu pour les pods. Par defaut, tous les pods dans un cluster Kubernetes peuvent communiquer entre eux. Les Network Policies permettent de restreindre ces communications.

Cas d'usage : Interdire a un pod malveillant ou compromis d'acceder directement a l'API. Seuls l'Ingress Controller et le frontend doivent pouvoir communiquer avec les pods flask-api.

```
# fichier: network-policy.yaml
# Restreint l'acces a l'API : seuls l'Ingress Controller et le frontend
# peuvent communiquer avec les pods flask-api.

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow-ingress-only
```

```yaml
spec:
  podSelector:
    matchLabels:
      app: flask-api              # S'applique aux pods flask-api
  policyTypes:
  - Ingress                       # Controle le trafic entrant
  ingress:
  # Regle 1 : Autoriser depuis le namespace ingress-nginx
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: ingress-nginx
  # Regle 2 : Autoriser depuis le frontend (meme namespace)
  - from:
    - podSelector:
        matchLabels:
          app: frontend
```

```bash
# Applique la Network Policy
kubectl apply -f network-policy.yaml

# Verifie que la politique est creee
kubectl get networkpolicies

# Teste : l'acces depuis l'Ingress devrait toujours fonctionner
curl http://monapp.local/api

# Teste l'isolation : un pod non autorise ne devrait PAS pouvoir acceder
kubectl run test-pod --image=busybox:1.28 --rm -it --restart=Never \
  -- wget -qO- http://flask-api-service --timeout=5
# Attendu : timeout (connexion refusee)
```

> **Note :** Les Network Policies necessitent un CNI compatible (Calico, Cilium...). Le CNI par defaut de minikube ne les enforce pas toujours. Si le test-pod recoit quand meme une reponse, c'est normal en environnement minikube de base.

Nettoyage (optionnel)

```bash
# Supprime toutes les ressources creees dans ce TP
kubectl delete deployment flask-api frontend
kubectl delete service flask-api-service frontend-service
kubectl delete ingress flask-api-ingress multipath-ingress 2>/dev/null || true
kubectl delete hpa flask-api flask-api-hpa 2>/dev/null || true
kubectl delete pod load-generator 2>/dev/null || true
kubectl delete networkpolicy api-allow-ingress-only 2>/dev/null || true

# Desactiver les addons (optionnel, gardez-les si vous refaites le TP)
# minikube addons disable ingress
# minikube addons disable metrics-server
```

Criteres de validation

Critere	Points
3 pods flask-api en cours d'execution	1 pt
Distribution de charge observable via curl	1 pt
Ingress Controller installe et fonctionnel	2 pts
Acces a l'API via /api sur monapp.local	2 pts
Rolling update effectue sans coupure	2 pts
Rollback reussi	1 pt
HPA configure avec min=2, max=5	2 pts
Scale-up declenche par le stress test	2 pts
Challenge : Frontend deploye + routing multipath	3 pts
Bonus : Network Policy isolant l'API	2 pts
Total	**18 pts (+2 bonus)**

Recapitulatif des concepts

Concept	Ressource K8s	Ce que ca fait
Scaling manuel	`kubectl scale`	Change le nombre de replicas immediatement
Scaling automatique	`HPA`	Ajuste les replicas selon la charge CPU
Load balancing	`Service`	Distribue le trafic entre les pods (round-robin)
Routage HTTP	`Ingress`	Dirige le trafic selon l'URL (path-based routing)
Mise a jour	`Rolling Update`	Remplace les pods un par un (zero downtime)
Retour arriere	`Rollback`	Revient a une version precedente via ReplicaSet
Self-healing	`ReplicaSet`	Recree automatiquement les pods en panne
Isolation reseau	`NetworkPolicy`	Pare-feu entre pods (qui peut parler a qui)

Troubleshooting

Probleme	Commande de diagnostic	Solution
Pod en `Pending` apres scale-up	`kubectl describe pod <pod>`	Ressources insuffisantes ; augmenter avec `minikube start --cpus=4 --memory=4096`
Ingress retourne `404 Not Found`	`kubectl describe ingress`	Verifier que le Service backend existe (`kubectl get svc`) et que les labels correspondent
`curl http://monapp.local/api` ne repond pas	`kubectl get pods -n ingress-nginx`	L'Ingress Controller n'est pas Running ; relancer `minikube addons enable ingress`
HPA affiche `<unknown>/50%`	`kubectl top pods`	metrics-server pas encore actif ; patienter ou relancer `minikube addons enable metrics-server`
Rolling update bloque (0 pods disponibles)	`kubectl rollout status deployment/flask-api`	Image invalide ou `ImagePullBackOff` ; faire `kubectl rollout undo deployment/flask-api`
load-generator en `Pending`	`kubectl describe pod load-generator`	Image `busybox:1.28` non dispo ; utiliser le manifest YAML alternatif fourni
Reponses identiques (meme pod) dans la boucle curl	`kubectl get endpoints flask-api-service`	Verifier que les 3 pods sont bien dans les Endpoints du Service
`watch` non reconnu (Windows/Mac)	—	Utiliser la boucle `while true; do ...; sleep 2; clear; done`

Annexe - Commandes de debogage utiles

```
# Voir les events du cluster (tres utile en cas de probleme)
kubectl get events --sort-by=.lastTimestamp

# Decrire un pod en erreur (events, conditions, configuration)
kubectl describe pod <nom-du-pod>
```

```
# Logs en temps reel d'un pod (-f = follow, comme tail -f)
kubectl logs -f <nom-du-pod>

# Entrer dans un pod pour tester la connectivite reseau
kubectl exec -it <nom-du-pod> -- /bin/sh

# Tester la resolution DNS interne depuis un pod temporaire
kubectl run dns-test --image=busybox:1.28 --rm -it --restart=Never \
  -- nslookup flask-api-service

# Verifier les endpoints d'un service (IPs des pods rattaches)
kubectl get endpoints flask-api-service

# Voir la configuration complete d'un ingress en YAML
kubectl get ingress multipath-ingress -o yaml
```